

UIT2739 – FULL STACK DEVELOPMENT

A PROJECT REPORT

on

LOST AND FOUND PORTAL USING MEAN STACK

Submitted by

SAI DESHNA BUDIDETI - 3122225002111

K V SWETHA - 3122225002143

B THEJESSWINI - 3122225002148



Department of Information Technology

Sri Sivasubramaniya Nadar College of Engineering

(An Autonomous Institution, Affiliated to Anna University)

Rajiv Gandhi Salai (OMR), Kalavakkam – 603 110

NOVEMBER 2025

**SRI SIVASUBRAMANIYA NADAR COLLEGE OF
ENGINEERING**



Department of Information Technology

CERTIFICATE

Certified that this project titled “Lost And Found Portal Using MEAN Stack” is the bonafide work of Sai Deshna Budideti (3122225002111), K V Swetha (3122225002143), B Thejesswini (3122225002148) and is submitted for project review on 27 November 2025.

Place : Kalavakkam

Date :

Internal Examiner

TABLE OF CONTENTS

1. Project Overview
2. Project Requirements
 - 2.1 Wireframes
 - 2.2 Functional Requirements & Use Cases
3. Technical Details
 - 3.1 Tech Stack
 - 3.2 Architecture Diagram
 - 3.3 Design Patterns
4. Output screenshots
5. Conclusion

1. PROJECT OVERVIEW

Problem Statement

College campuses frequently face issues where students lose or misplace personal belongings such as ID cards, wallets, water bottles, and stationery items. There is typically no centralized digital system for reporting lost/found items, which causes inconvenience and delays in recovery.

This project, Lost & Found Portal, offers a web-based community platform designed specifically for campus environments. Students can upload lost or found items, providing details and images. Other users can search, contact, and claim items easily.

This ensures:

- Quick recovery of lost belongings.
- A centralized communication channel within campus
- User accountability via login (JWT-based authentication)

The system is scalable and can be adapted for institutions, public places, and events.

Proposed Solution

A MEAN stack web portal allows students and staff to:

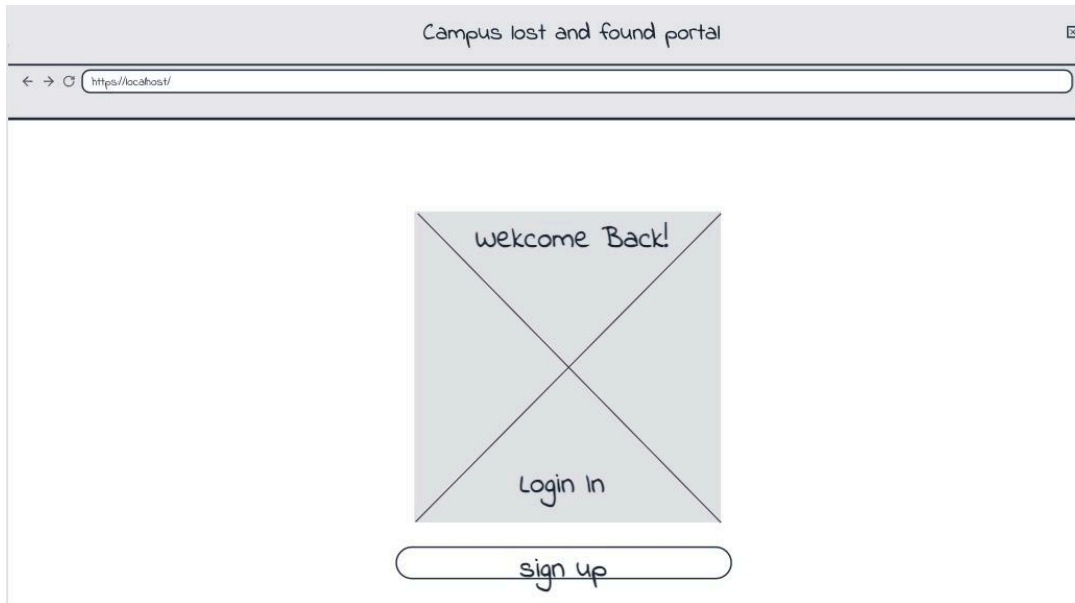
1. Report an item as *lost*, *found*, or *found & returned*
2. Upload supporting images
3. View/search available items
4. Update or delete their own submitted items
5. Secure login using JWT Authentication

This enhances efficiency, transparency, and community-driven retrieval.

2. PROJECT REQUIREMENTS

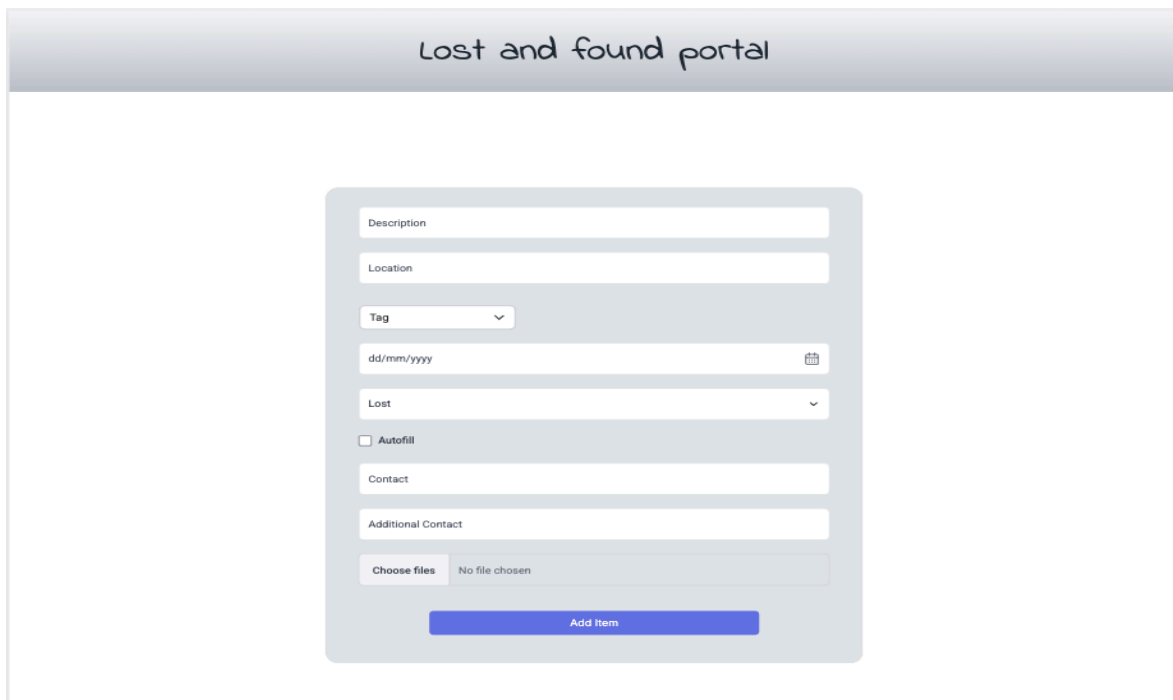
2.1 Wireframes

a. Login/ Sign Up



A wireframe for a web browser window titled "Campus lost and found portal". The address bar shows "https://localhost/". The main content area features a large gray square with a black 'X' across it. The text "wekcome Back!" is in the top-left triangle and "Login In" is in the bottom-right triangle. Below the square is a rounded rectangular button labeled "sign up".

b. Add item form



A wireframe for a web browser window titled "Lost and found portal". The main content area displays a form for adding a new item. The form fields are: "Description", "Location", "Tag" (a dropdown menu), "dd/mm/yyyy" (a date input field with a calendar icon), "Lost" (a dropdown menu), an "Autofill" checkbox, "Contact", "Additional Contact", and a file upload section with a "Choose files" button and "No file chosen" text. At the bottom of the form is a blue "Add Item" button.

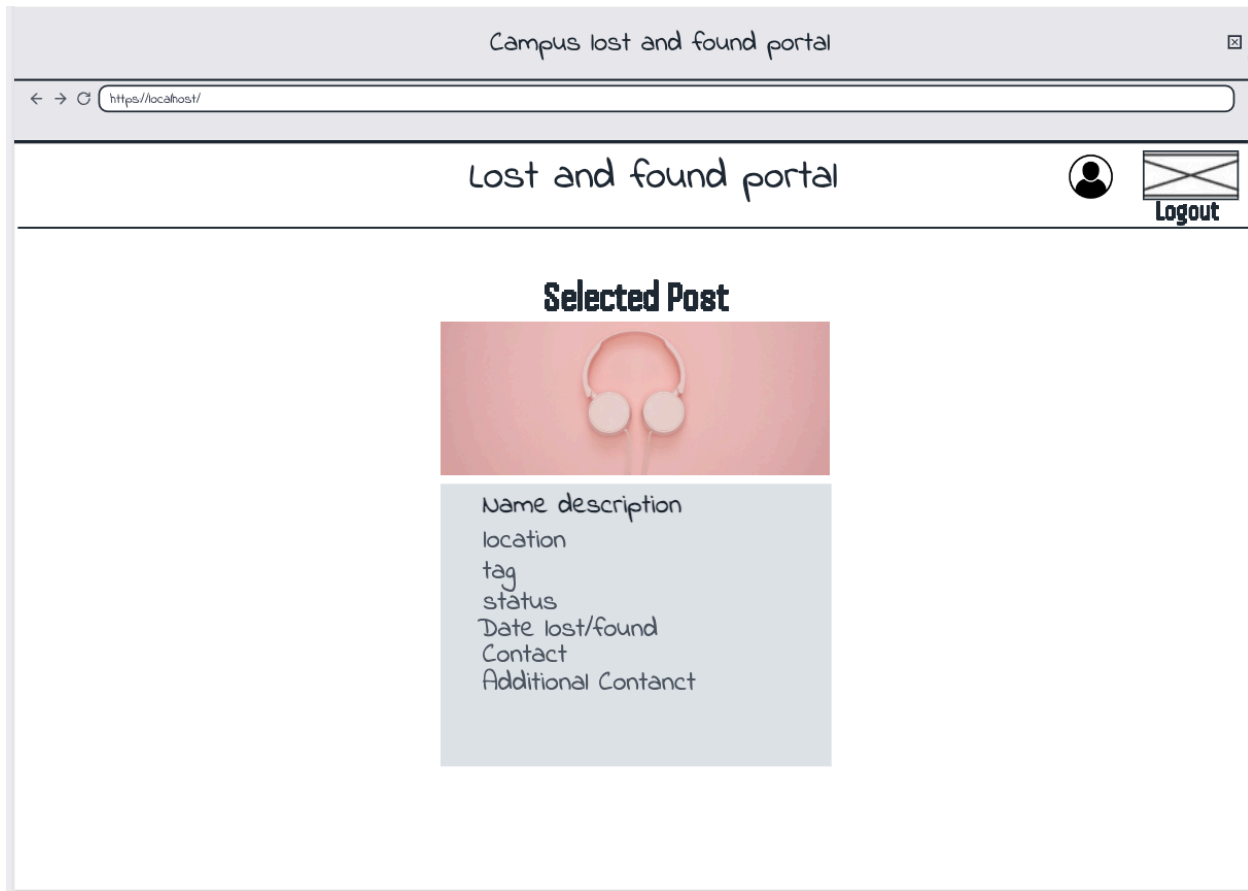
c. Update section

The screenshot shows a web browser window with the address bar set to `https://localhost/`. The page title is "Lost and found portal". In the top right corner, there is a user profile icon and a "Logout" button. Below the header, there is a "User Profile" button. The main content area displays three identical item cards. Each card has a blue header, a "Description" input field, and two buttons: "UPDATE" and "DELETE".

d. Homescreen

The screenshot shows a web browser window with the address bar set to `https://localhost/`. The page title is "Lost and found portal". In the top right corner, there is a user profile icon and a "Logout" button. Below the header, there is a section titled "Add New Item Form" which contains a large gray box with a diagonal cross. Below this, there is a section titled "List of Items" which contains a search input field and four item cards. Each card has a placeholder image (a box with a diagonal cross) and a list of fields: "Name", "description", "location", "tag", and "status".

e. Selected card display



2.2 Functional Requirements

The Lost & Found Portal is designed to support a seamless and secure process for reporting, managing, and retrieving lost belongings within a campus environment. To ensure a user-friendly experience and reliable item recovery, the system incorporates essential functional features such as secure user authentication, detailed item submissions, efficient browsing and searching, and update capabilities.

These features enable students to easily upload lost or found items, track item status, and communicate with item owners through contact information. With built-in role protection and validation via JWT-based authentication, users can confidently manage only their own posts, ensuring safety and accountability.

Feature	Description
User Authentication	Users must login/register to add items (JWT token-based)
Add Lost/Found Item	Upload image, enter item details, location, date, contact information
Edit / Update Item	Users can update details of their own posts
Delete Item	Users can remove their uploaded entries
View Items	Display all lost/found items and upon clicking the item, one can look at the item details
Autofill	Automatically fills the contact details of the user
Filter & Search	Search by item category (tag)
Item Status Update	Mark item as "Found", "Lost", "Found and Returned"

2.2 Use Cases

Use cases describe how different types of users interact with the system to achieve their goals. In the Lost & Found Portal, students play the primary role as contributors and

searchers of lost belongings, while the system acts as a secure platform that manages and stores item information.

These use cases help define user interactions such as reporting a lost item, posting a found item, searching for items, and updating item status. Each use case outlines the flow of actions, constraints, and outcomes that ensure the application functions efficiently while maintaining data security and user authentication. By analyzing these interactions, we validate that the system fulfills real-world requirements within a campus environment.

Actor	Use Case	Description
Student	Login	Authenticate using username/password
Student	Add Item	Report Lost/Found item with images
Student	Edit/Delete Item	Manage personal posts
Student	Search Item	Locate an item based on filters
Student	Contact Owner	Use phone/email/Instagram to claim item
Student	Autofill	Automatically fills the contact details

3. TECHNICAL DETAILS

Layer	Technology Used
Frontend	Angular 17, TypeScript, HTML, SCSS
Backend	Node.js, Express.js
Database	MongoDB with Mongoose ODM
Authentication	JWT (JSON Web Token)
Deployment (optional)	Localhost / Cloud server
Image Storage	Stored as Base64 in database (current implementation)

3.1 Tech Stack Explanation

The Lost & Found Portal is built using the **MEAN stack**, a popular full-stack architecture that supports rapid and scalable web application development. Each layer plays a specific role in handling the flow of data, user interaction, and security. The technologies used in this project are listed and explained below.

Frontend – Angular 17, TypeScript, HTML, SCSS: The frontend of the application is developed using **Angular 17**, a component-based framework provided by Google.

3.1.1 Advantages of Angular

- **Two-way data binding** ensures real-time synchronization between UI and data model.
- **Standalone components** (introduced in recent versions) simplify module dependency.
- **Routing support** allows navigation between pages such as:
- **Form validation** for inputs like contact numbers, date, and required fields.

3.1.2 TypeScript

Angular uses **TypeScript**, which provides:

- Better developer experience through **static typing**
- Error identification during development rather than runtime
- Cleaner and more maintainable code

3.1.3 HTML + SCSS

- **HTML** forms the structure and layout of the UI.
- **SCSS** allows modular and responsive styling for:
 - Cards for displaying items
 - Upload form layout
 - Search bar and filters



3.1.4 Backend

Node.js: Node.js provides a **JavaScript runtime environment**, enabling server-side logic using the same language as the frontend.

Express.js: A lightweight web framework used to:

- Define REST API endpoints (GET, POST, PUT, DELETE)
- Handle routing such as:
 - `/api/items` → Add or view items
 - `/api/auth/login` → Generate token upon login



3.1.4 Database – MongoDB + Mongoose ODM

MongoDB (NoSQL database) stores application data in flexible **JSON-like documents**.

Need for MongoDB:

- Item records vary (e.g with or without images), so a **schema-less database** is ideal.
- Fast and optimized for **read-heavy** operations like searching for lost items.

Mongoose ODM

- Defines data structure using **schema** (Item, User, Tags)
- Automatically converts JavaScript objects into MongoDB documents
- Provides query functions like find(), findById(), save(), etc.



Structure Schema of Item: JSON format

```
{ "__id": "68f781f9e9dca60d9c5c4b03",  
  "images": [ // ... image data ... ],  
  "description": "Black Wallet Leather with ID card inside",  
  "location": "CT",  
  "tag": "wallet",  
  "dateLost": "2025-11-08T00:00:00.000Z",  
  "status": "lost",  
  "autofill": false,  
  "contact": "8999003023",  
  "additionalContact": "",  
  "itemId": "68f781f9e9dca60d9c5c4b04",  
  "createdAt": 2025-11-25T17:55:35.668+00:00
```

```
"updatedAt":2025-11-25T17:55:35.668+00:00  
"__v": 0}
```

3.1.5 Authentication – JSON Web Token (JWT)

Security is critical because users must control only their own posts.

How JWT works in this project:

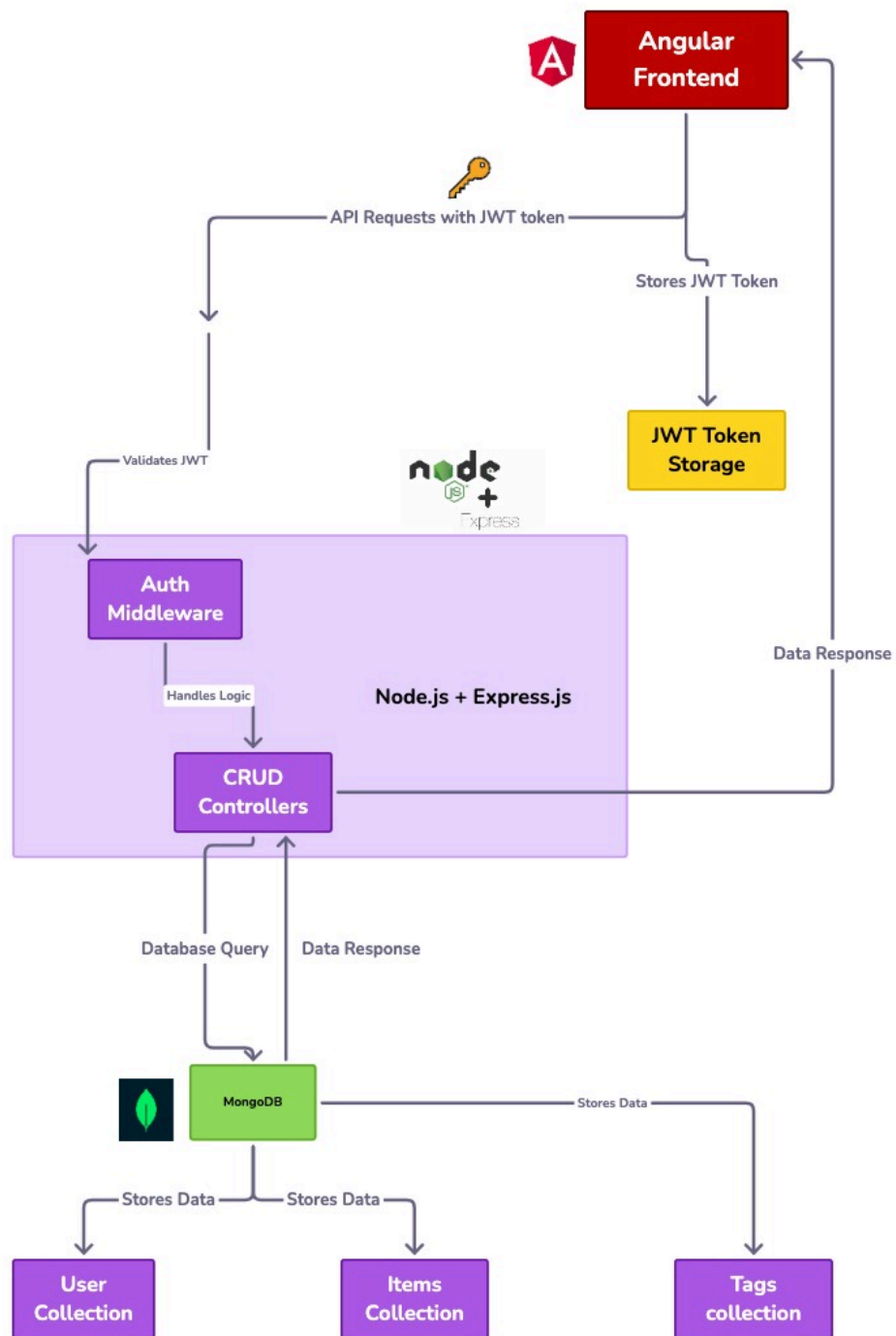
1. User logs in with credentials
2. Backend validates user and returns a **signed token**
3. Angular stores this token in localStorage
4. For protected operations like Edit/Delete item, token is sent in headers
5. Server verifies token and grants access

Advantages:

- **Stateless authentication** — no need for server-side session storage
- Prevents unauthorized edits or fake posts
- Ensures user identity is verified in each request



3.2 Architecture Diagram



The Lost & Found Portal is built using the MEAN Stack (MongoDB, Express, Angular, Node.js). Users interact with the system through the Angular frontend where they can login and manage lost or found items. When a user logs in, the backend verifies their credentials and generates a JWT session token. This token is stored in the browser and is automatically attached with every protected API request.

Node.js with Express receives these requests, validates the token for security, and then communicates with MongoDB using Mongoose to perform operations such as adding items, retrieving all items, updating details, or deleting posts. If the token expires or is invalid, the user must log in again. Images are stored in the database in Base64 format along with item details. Search and filter features help users quickly find lost or found items.

Thus, the system ensures secure item management, easy accessibility, and smooth communication between the frontend, backend, and database.

3.3. Design Patterns

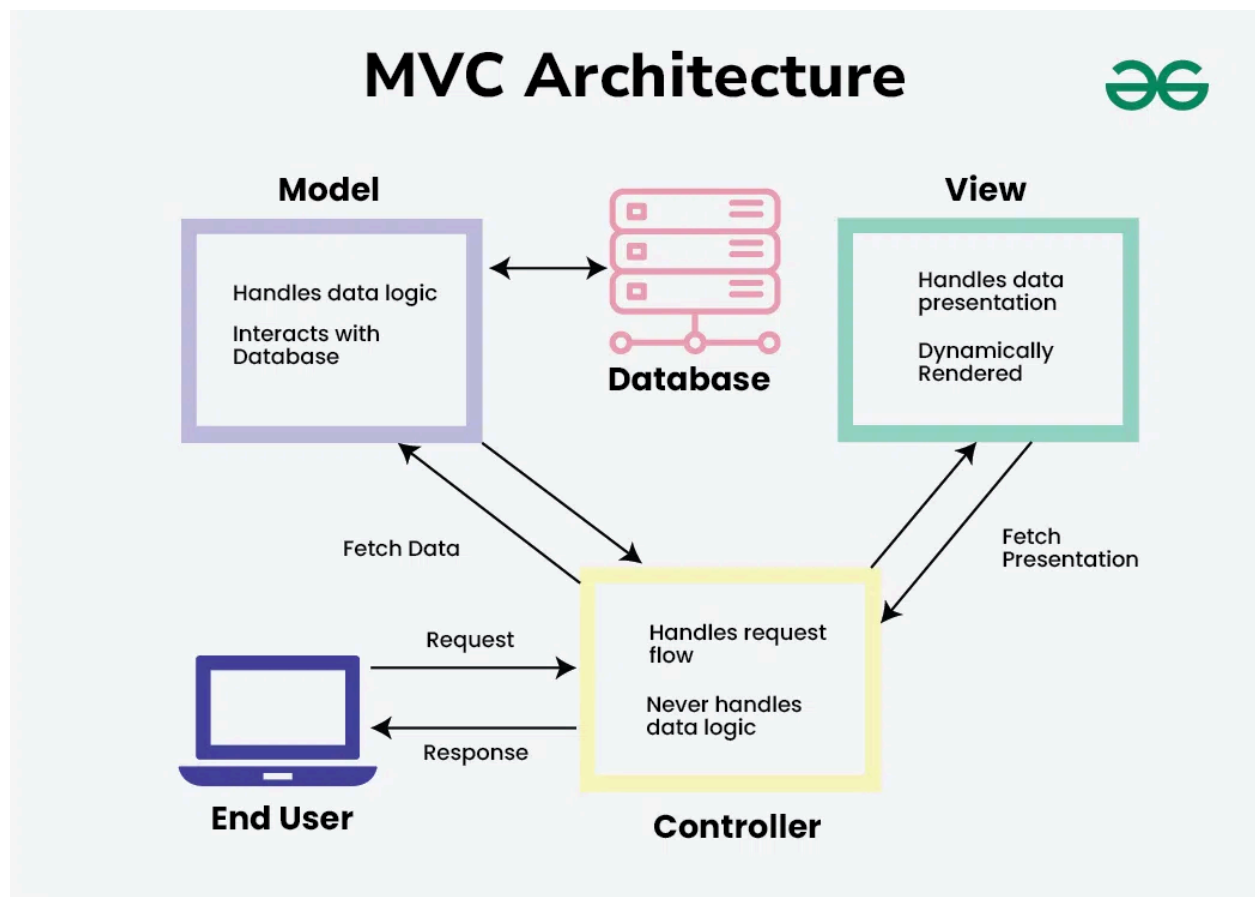
The Lost & Found Portal, developed using the MEAN stack (MongoDB, Express.js, Angular, Node.js), adopts multiple established software design patterns that improve the structure, scalability, and maintainability of the application. These patterns ensure clean separation of concerns, secure communication, and consistent behavior across various layers of the system.

1. MVC Pattern (Model–View–Controller)

The backend architecture follows the MVC design pattern.

- Models represent the structure of application data — such as Users and Items using Mongoose schemas.
- Views are not directly handled by the backend (since Angular takes care of UI), but the API returns JSON responses consumed by the frontend.
- Controllers handle incoming HTTP requests, execute business logic, and interact with models.

This separation allows independent development of the database, API logic, and user interface, leading to better modularity and easier updates.



2. Singleton Pattern (Database Connection Management)

The Singleton Pattern is used in the backend for establishing the MongoDB connection. Mongoose creates one shared instance of a database connection for the entire application. This avoids multiple unnecessary connections, reduces memory usage, and ensures consistent interaction with the database throughout the lifecycle of the server.

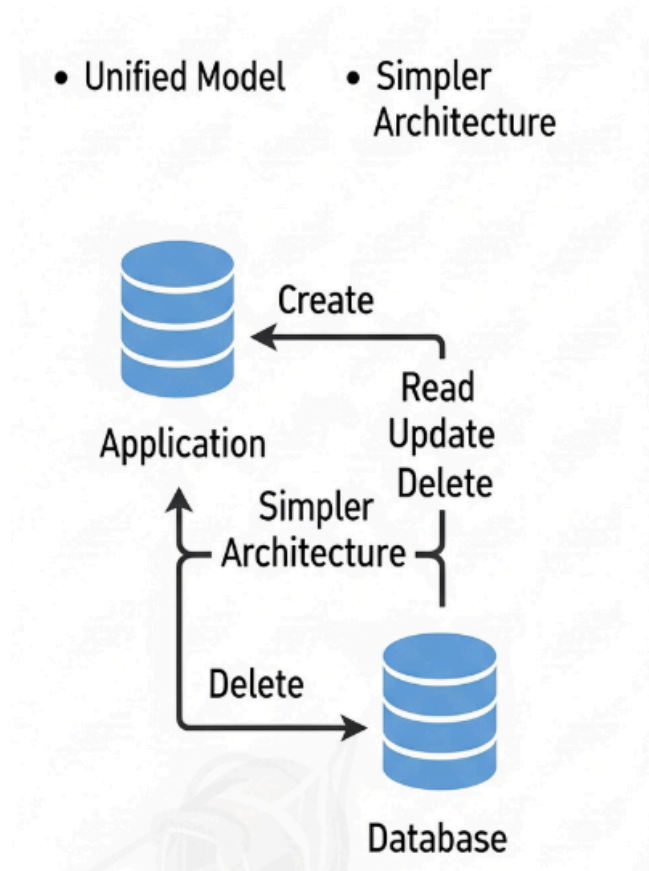
3. CRUD Design Pattern

The entire portal is structured around Create, Read, Update, and Delete operations. Users can:

- Create new lost/found item posts

- Read/view the item list
- Update their own posts (e.g., change status to Returned)
- Delete posts when resolved

CRUD ensures that the system remains simple, understandable, and aligned with RESTful API principles.



4. OUTPUT SCREENSHOTS

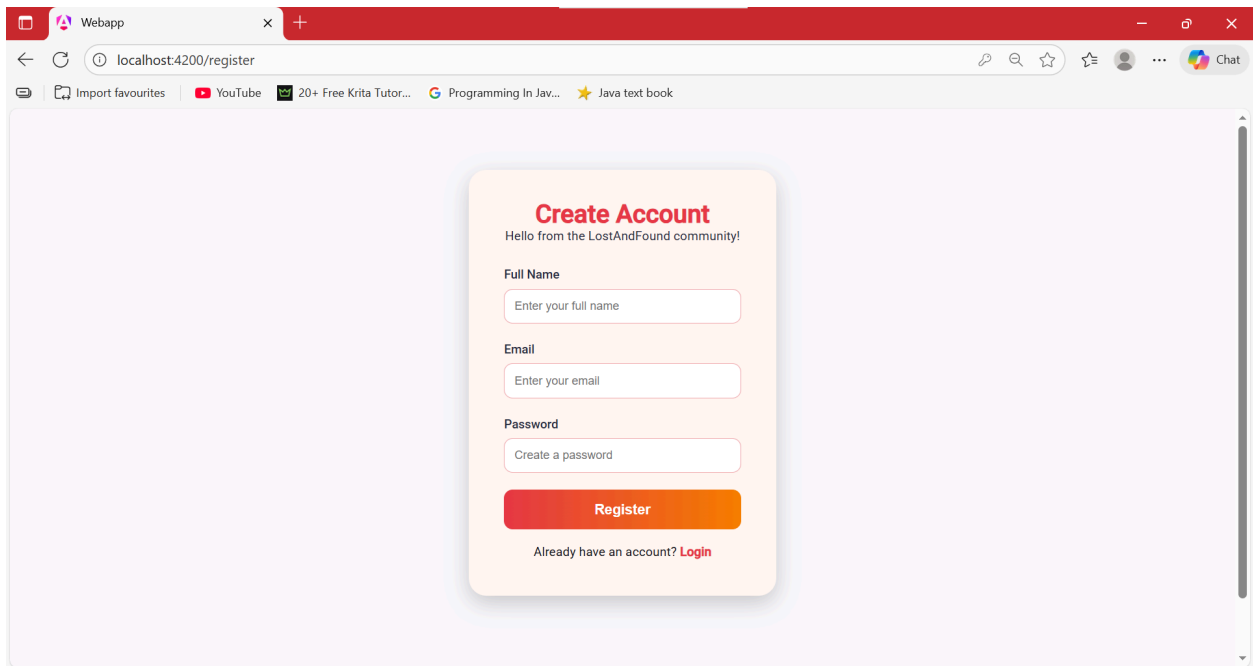
Login page

The screenshot shows a web browser window with the address bar displaying 'localhost:4200/login'. The page features a light purple background. In the center, there is a white rounded rectangle with a subtle shadow. At the top of this rectangle, the text 'Welcome Back' is written in bold red, followed by 'Login to continue' in a smaller black font. Below this, there are two input fields: 'Email' with the placeholder 'Enter your email' and 'Password' with the placeholder 'Enter your password'. An orange 'Login' button is positioned below the password field. At the bottom of the white rectangle, it says 'Don't have an account? [Sign up](#)'.

Login page validation

This screenshot shows the same login page as before, but with validation feedback. The 'Email' field now contains the text 'test@example' and the 'Password' field is filled with dots. The orange 'Login' button remains visible. At the bottom of the browser window, a dark grey message bar has appeared with the text 'Invalid username or password.' and a 'Close' button on the right.

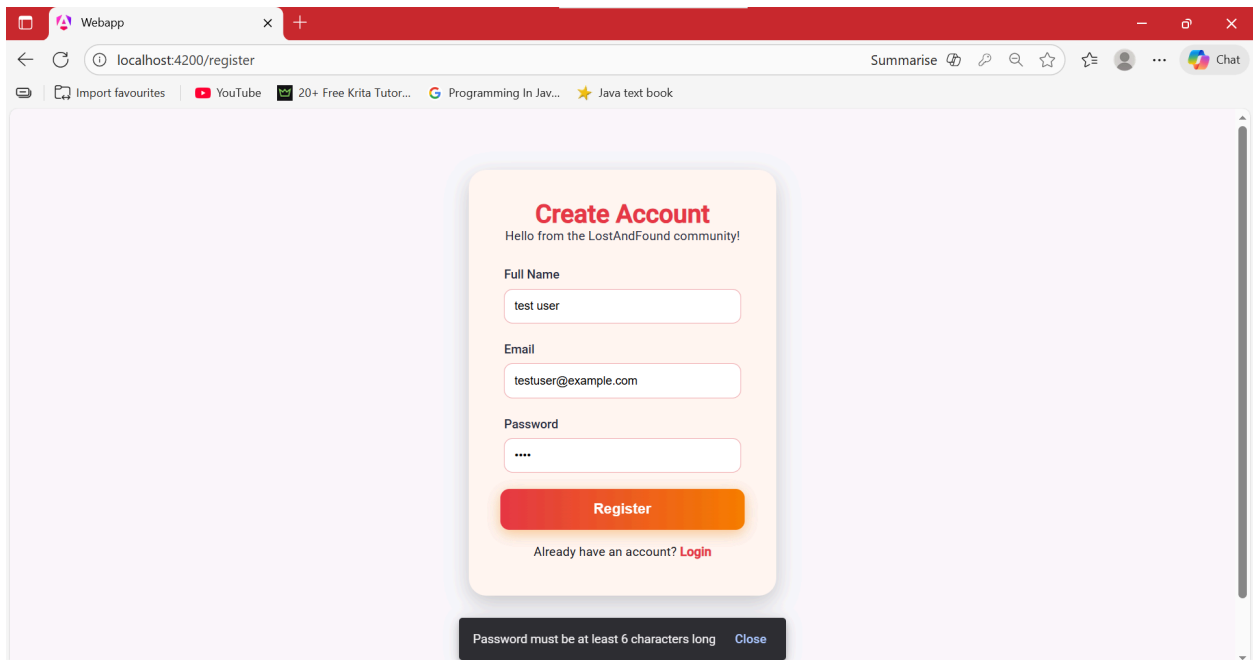
Registration page



The screenshot shows a web browser window with the address bar displaying "localhost:4200/register". The page features a central "Create Account" form with the following elements:

- Create Account** (Section Header)
- Hello from the LostAndFound community!
- Full Name** (Label) with an input field containing the placeholder text "Enter your full name".
- Email** (Label) with an input field containing the placeholder text "Enter your email".
- Password** (Label) with an input field containing the placeholder text "Create a password".
- Register** (Orange Button)
- Already have an account? [Login](#)

Registration page validation



This screenshot shows the same registration page as above, but with test data entered and a validation message displayed:

- Full Name** input field contains "test user".
- Email** input field contains "testuser@example.com".
- Password** input field contains "****".
- The **Register** button is still visible.
- A dark notification bar at the bottom of the form area displays the message: "Password must be at least 6 characters long" with a **Close** button.

Home page (after successful registration)

The screenshot shows a web browser window with the URL `localhost:4200`. The page has a red header bar with the text "Lost and Found Portal" and a "Logout" button. Below the header, there is a form for adding a new item. The form fields are: Description, Location, a dropdown menu with "water bottle" selected, a date field with "mm/dd/yyyy" placeholder, a dropdown menu with "Lost" selected, an "Autofill" checkbox, a Contact field, an Additional Contact field, and a "Choose Files" button. A message at the bottom of the form says "User registered! Welcome to LostAndFound" with a "Close" button.

Adding an item

The screenshot shows the same web browser window as before, but the "Add Item" button at the bottom of the form is highlighted in red. The form fields are filled with: "brown wallet" for Description, "near fountain" for Location, "wallet" for the dropdown menu, "11/24/2025" for the date field, "Lost" for the dropdown menu, the "Autofill" checkbox is checked, "7859032834" for the Contact field, and an empty "Additional Contact" field. The "Choose Files" button is still present.

The screenshot shows the same web browser window, but a modal dialog box is displayed in the center. The dialog box has a green checkmark icon and the text "Success! Item added successfully". There is an "OK" button at the bottom of the dialog box. The background of the page is dimmed.

Autofill - Automatically fills the contact details (Email) of user

The screenshot shows a web browser at localhost:4200 displaying the 'Lost and Found Portal'. A form titled 'Add Item' is centered on the page. The form fields are: 'Flower structure earring', 'CT near Ashwini', 'Precious Item' (dropdown), '14/11/2025' (date), 'Lost' (dropdown), 'Autofill' (checked checkbox), 'kvswe04@gmail.com' (email field, auto-filled), 'Additional Contact', and a file upload button. A red 'Add Item' button is at the bottom of the form.

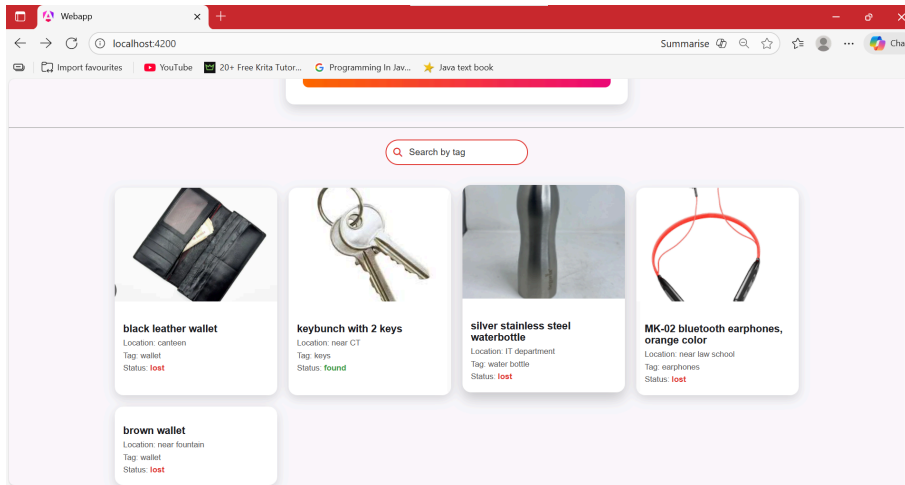
Validation to ensure the required fields are filled

The screenshot shows the 'Lost and Found Portal' with the 'Add Item' form. The 'Autofill' checkbox is now unchecked. The form fields are: 'brown wallet', 'near fountain', 'wallet' (dropdown), '11/24/2025' (date), 'Lost' (dropdown), 'Autofill' (unchecked), 'Contact' (empty), 'Additional Contact', and a file upload button. A red error message at the bottom of the form reads: 'Please fill in Description, Location, and Contact'.

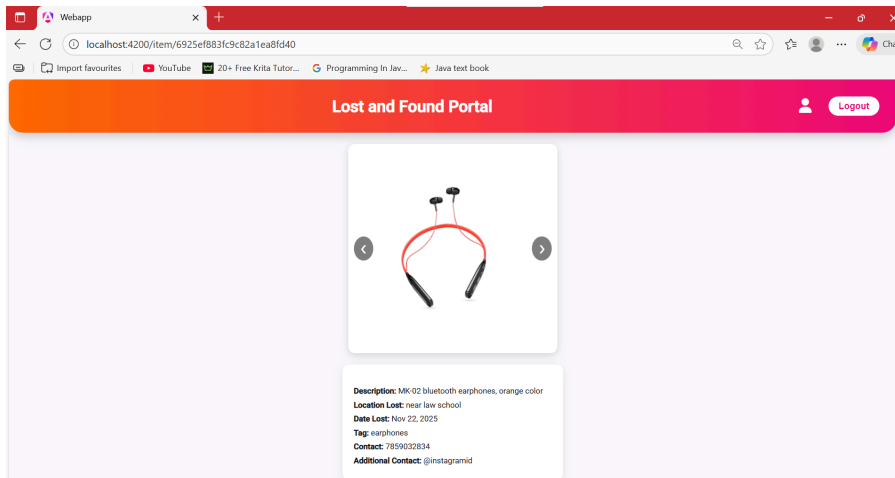
Validation for date

The screenshot shows the 'Lost and Found Portal' with the 'Add Item' form. The form fields are: 'brown wallet', 'near fountain', 'water bottle' (dropdown), '11/28/2025' (date), 'Lost' (dropdown), 'Autofill' (unchecked), '7859032834' (phone number), 'Additional Contact', and a file upload button. A red error message at the bottom of the form reads: 'Date lost/found cannot be in the future'.

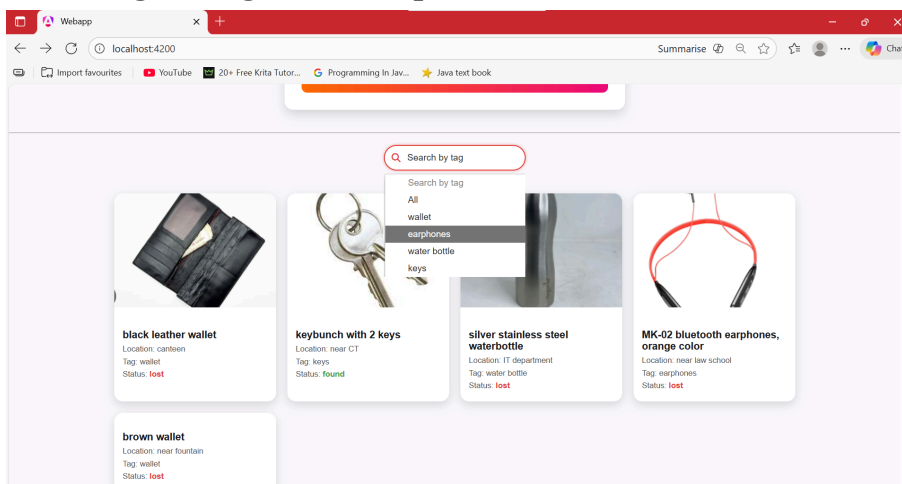
Viewing posts

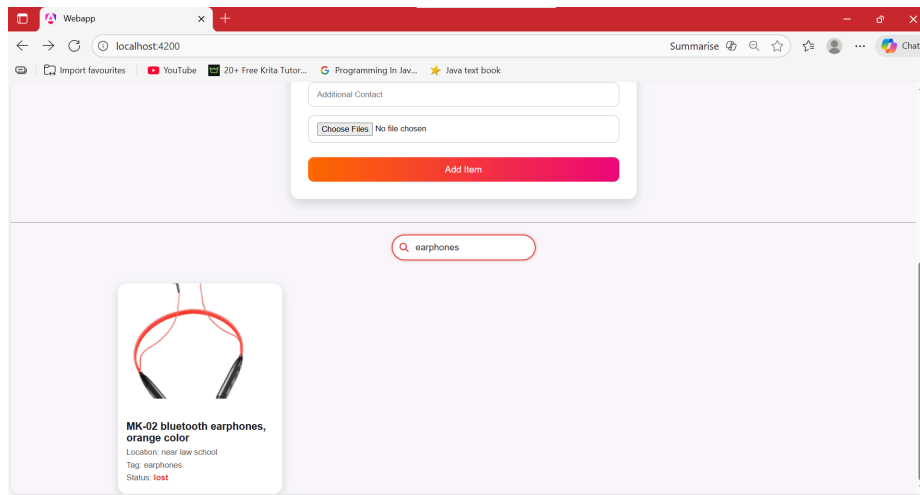


Viewing the details for a particular post

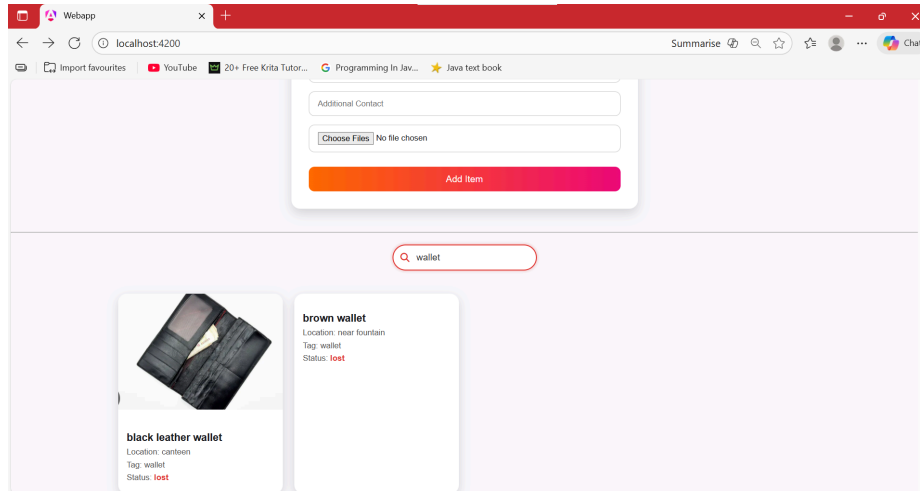


Searching via tags → 1. Earphones:

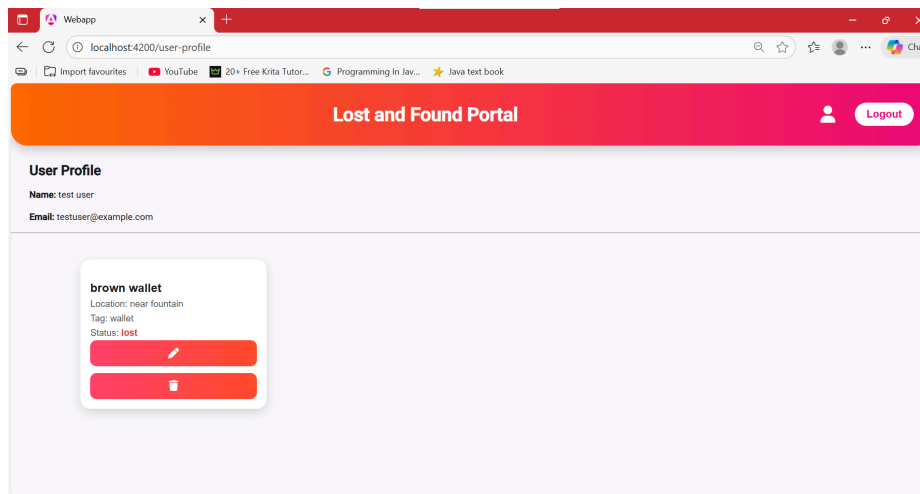




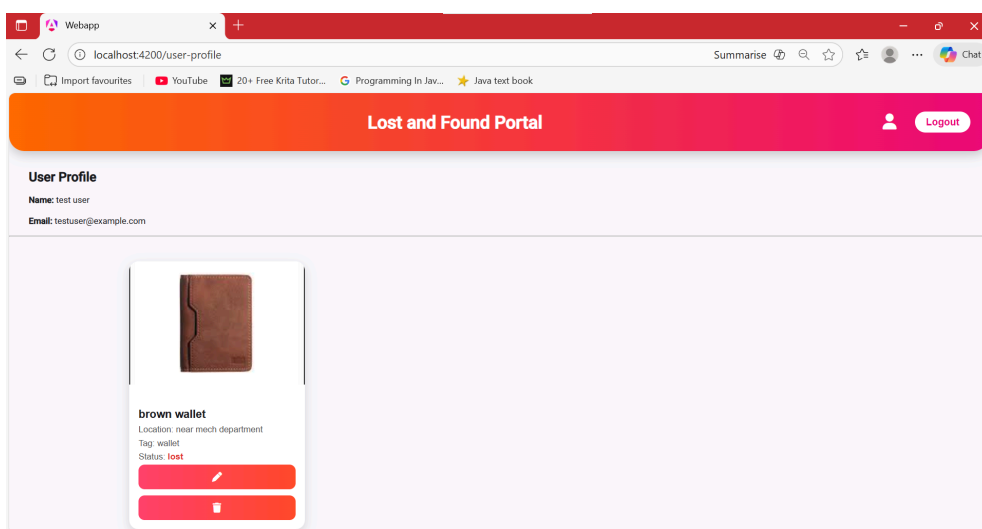
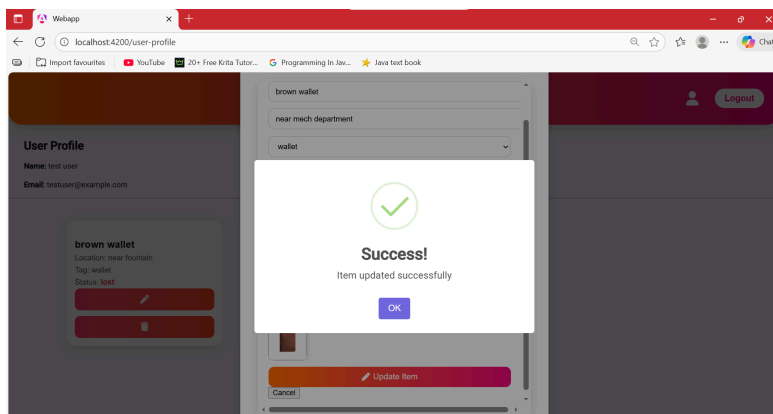
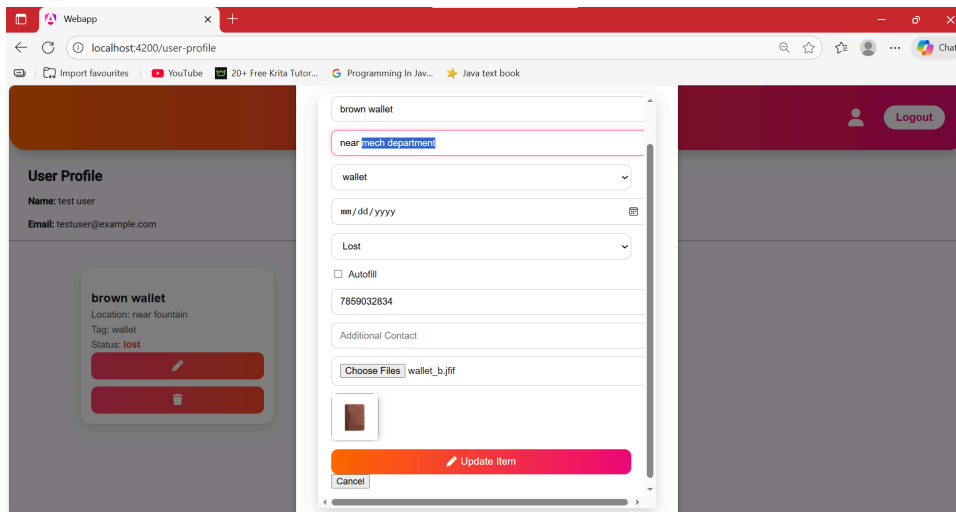
Searching via tags —> 2. Wallet:



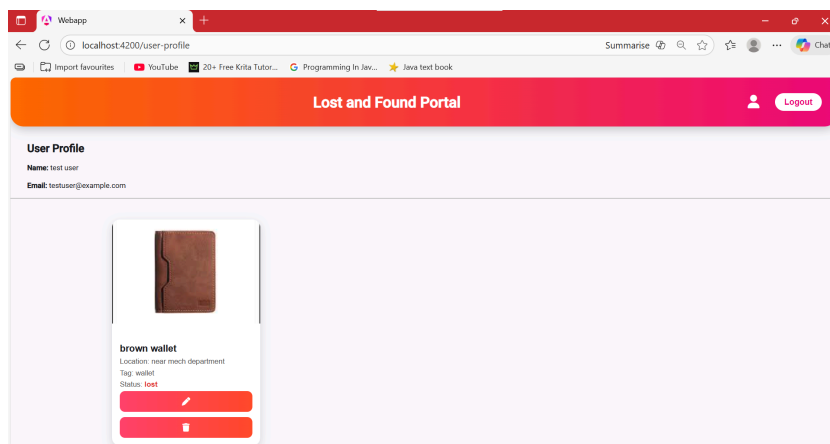
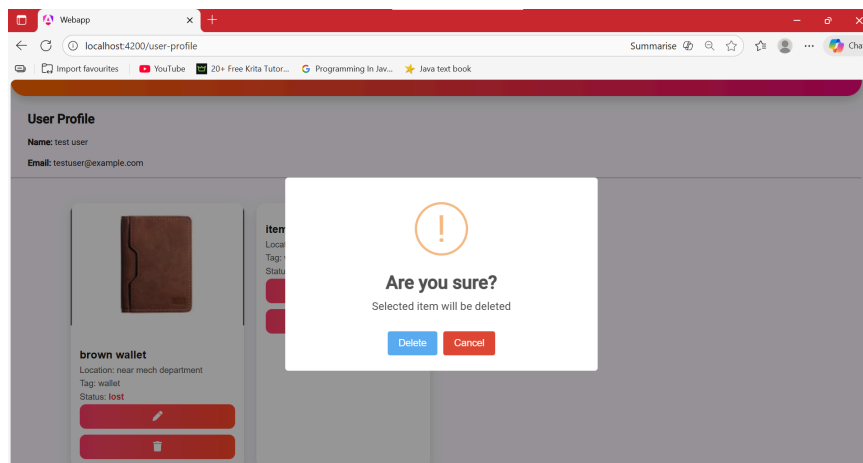
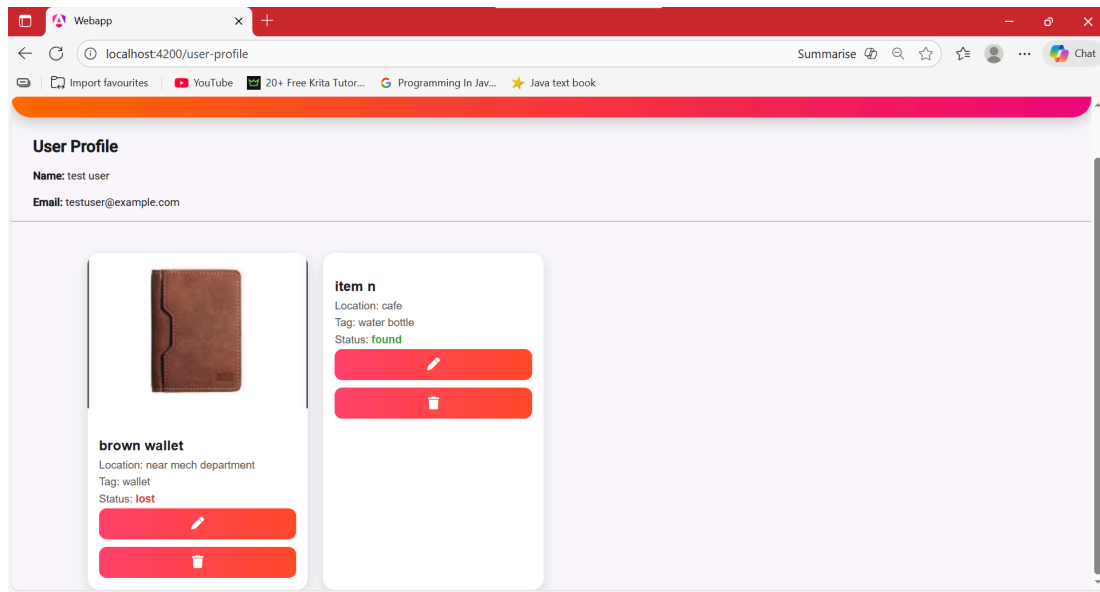
Viewing user profile



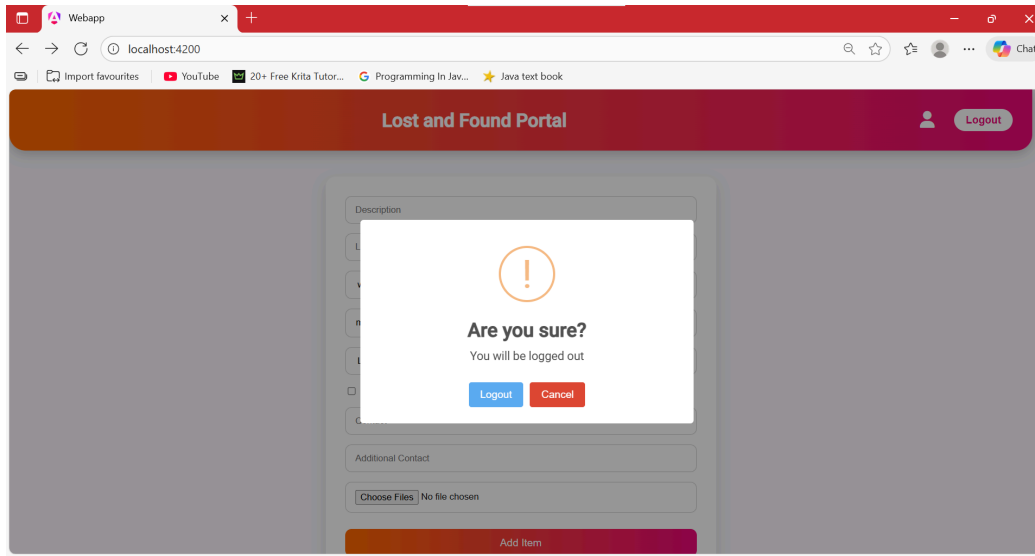
Editing a post



Deleting a post



Logout



5. CONCLUSION

The Campus Lost & Found Portal successfully provides a centralized and secure digital solution to track misplaced items within the college environment. By utilizing the MEAN stack, the system ensures fast performance, scalable data handling, and a modern user experience. Features like user authentication, secure CRUD operations, image uploads, and search capability make the platform practical and highly useful for real-world campus scenarios. The use of JWT enhances data security, ensuring only authorized users manage item listings. Overall, this project demonstrates how web technologies can effectively solve everyday problems and promote a more connected and responsible campus community.